
EVALUATING I/O BACKENDS FOR LLM CHECKPOINT LOADING ACROSS FORMATS, LAYERS, AND ENVIRONMENTS

Botir Khaltayev
Independent, United Kingdom
btrghstk@gmail.com

ABSTRACT

Cold-start latency in large language model (LLM) serving is usually treated as a bandwidth problem: how quickly can storage stream checkpoint bytes to GPU memory? This paper argues that the dominant factor is not peak throughput but the interaction between *checkpoint layout*, *I/O backend*, and *integration layer*. A checkpoint layout determines the file and range requests exposed to the operating system; an I/O backend determines how those requests are submitted and completed; Python bindings and serving engines determine how much of any backend gain reaches the user. Consequently, no single I/O backend dominates across the full parameter space.

We present a systematic evaluation using Tensora, an open checkpoint-loading framework designed for backend attribution. We compare sync, async (Tokio), and Linux `io_uring` across two layouts (contiguous SafeTensors shards and a partitioned ServerlessLLM-style layout), six public checkpoints from 0.6B to 32B parameters, two measured layers (Rust backend attribution and vLLM integration), and four environments (H100 with local NVMe, H200 and A100 with blocked `io_uring`, Modal H100 under gVisor). Held-out validation on Mistral-7B-Instruct confirms that regime structure generalises beyond the primary model ladder. Representative single-run screening on H100, H200, and A100 provides broad regime coverage; repeated five-repetition anchoring on the Modal H100 environment provides complementary variance evidence in a restricted container setting.

Four findings emerge. First, H100 SafeTensors loading exhibits a scale crossover: sync is fastest for small models (0.6B to 4B); `io_uring` is fastest from 8B through 32B (1.28× over sync at 32B). Second, on H100, sync is never the fastest backend on the ServerlessLLM layout; async and `io_uring` trade places by partition structure and scale. Third, integration layers reorder absolute times but preserve relative backend effects: in vLLM cold `load_only`, Tensora’s `io_uring` SafeTensors path reduces initialization by 37 to 48% for 8B and 32B checkpoints relative to vLLM’s native loader. Fourth, ordering regimes are more portable than backend identities: small-scale sync dominance transfers across environments, but the large-scale winner depends on I/O topology and kernel capability.

Keywords large language models · checkpoint loading · I/O backends · `io_uring` · cold start · vLLM · systems evaluation

1 Introduction

An LLM serving request cannot generate a token until model weights are resident. In serverless and multi-tenant settings, idle models are commonly evicted to recover scarce accelerator memory; the next request then pays a cold load before inference begins. A 7B float16 checkpoint is already on the order of 14 GB, and the checkpoints in this paper reach beyond 65 GB of weight bytes. Cold loading is therefore not a startup footnote; it is part of the user-visible serving path.

The usual shorthand is to call this a bandwidth problem. That shorthand is incomplete. A loader does not read an abstract number of gigabytes; it executes a concrete read plan. The checkpoint format decides whether bytes appear as a few contiguous shards or as many indexed ranges across partition files. The I/O backend decides whether those reads are submitted through blocking POSIX calls, an async runtime, or `io_uring`. The integration layer then adds its own costs: Python object construction, tensor materialization, CUDA context setup, allocator initialization, and serving-engine configuration. A device can have high peak throughput and still produce poor cold-start time when request geometry and submission method are mismatched.

Existing LLM serving systems and checkpoint libraries typically treat loading as a fixed initialization cost with a single code path. No prior work, to our knowledge, systematically evaluates I/O backend choice for LLM checkpoint loading as a function of format, scale, and deployment environment. This gap matters because the optimal backend is not stable across the parameter space.

This paper evaluates three explicit I/O backends: synchronous POSIX I/O, Tokio-based async loading, and Linux `io_uring`, across two checkpoint layouts, six model scales, four environments, and two measured layers (Rust and vLLM) with Python as the integration bridge. Our central finding is:

No single I/O backend dominates. The best backend depends on checkpoint layout, model scale, backend availability, and the layer at which loading is observed. A fixed rule such as “always sync,” “always async,” or “always `io_uring`” leaves performance on the table.

SafeTensors presents mostly whole-file or multi-shard eager loading: after metadata parsing, the loader must move large contiguous byte regions. ServerlessLLM-style partitioning presents a different workload: tensors are reconstructed from indexed partition files, creating many grouped range reads. These layouts are not cosmetic alternatives; they expose different I/O regimes. Scale then moves the boundary within a regime: a backend with low orchestration overhead can win for a small single-shard load, while a backend that sustains deeper submission queues can win for a large multi-shard load.

We built Tensora¹, an open framework for evaluating these regimes. Tensora exposes sync, async, and `io_uring` under shared format abstractions, enabling controlled backend attribution across layers. We evaluate six public checkpoints from 0.6B to 32B parameters across two layouts and two measured layers: Rust backend attribution and a custom vLLM integration. The primary environment is an H100 host with local NVMe and working `io_uring`; H200 and A100 container environments with network-attached storage and blocked `io_uring` test whether ordering regimes transfer when the strongest H100 backend is unavailable. A Modal H100 environment under gVisor provides restricted-container validation.

On H100 SafeTensors, sync wins at the small end and `io_uring` from Qwen3-8B through Qwen3-32B (1.28× over sync). On H100 ServerlessLLM, sync is never fastest; async and `io_uring` lead different rows. vLLM reorders backend rankings (it measures a different construct) but the effect survives integration: in vLLM cold `load_only`, Tensora’s `io_uring` SafeTensors path reduces initialization by 37 to 48% relative to vLLM’s native loader for 8B and 32B checkpoints. Cross-environment results preserve small-scale sync dominance on three of four environments while showing that the large-scale winner depends on kernel and storage context.

Contributions.

1. **Format-driven backend dependence.** We evaluate three I/O backends (sync, async, `io_uring`) across two checkpoint layouts and six model scales, showing that the best backend shifts systematically with format and scale: sync wins on small SafeTensors, `io_uring` wins on large SafeTensors, and sync never wins on H100 ServerlessLLM. To our knowledge, this is the first controlled evidence that LLM checkpoint loading is workload-dependent rather than bandwidth-bound.
2. **Backend regime persistence through integration.** We provide controlled evidence across two measured layers (Rust and vLLM) demonstrating that backend ordering regimes propagate through library and serving-engine boundaries. The proportional advantage shrinks but the regime structure survives.
3. **Regime transfer across environments.** We validate across four environments (H100, H200, A100, Modal) that small-scale sync dominance transfers widely while the large-scale winner depends on storage topology and kernel capability. Regime structure, not backend identity, is the portable quantity.

¹Tensora is this paper’s checkpoint-loading framework (Rust crate `tensora`, Python package `tensora_py`). It is unrelated to Google’s TensorStore library for array storage.

2 Related Work

Tensora sits between four lines of work: LLM serving engines, serverless cold-start systems, checkpoint formats and loading infrastructure, and operating-system I/O interfaces. Prior work either optimizes the serving pipeline after weights are resident, avoids cold loads through placement and caching, changes the checkpoint/storage substrate, or improves kernel-level page service. Tensora targets the application-level loading decision: given a concrete checkpoint layout and host capability set, which userspace backend should execute the read plan?

2.1 LLM serving systems

Modern serving systems primarily optimize inference after weights are resident. vLLM introduces Page-dAttention for KV-cache management and high-throughput serving [1]. Orca uses iteration-level scheduling [2]. Sarathi-Serve studies the prefill/decode scheduling trade-off [3]. DeepSpeed Inference applies model parallelism and kernel optimizations [4]. SGLang optimizes structured generation with RadixAttention and efficient runtime scheduling [5]. These systems make the steady-state serving pipeline efficient, but typically treat checkpoint loading as a fixed initialization cost.

Our vLLM integration is therefore a boundary test. Rust measurements explain backend mechanics; vLLM measurements test whether those mechanics survive in an engine with CUDA setup, memory pools, runtime configuration, and generation state. The contribution is not a new serving scheduler. It is a backend-evaluation framework whose deployment impact can be observed inside a serving engine.

2.2 Serverless and cold-start mitigation

Serverless LLM systems make cold start a first-order concern because evicted models must be restored on demand. ServerlessLLM [6] proposes multi-tier loading across GPU, CPU memory, and storage. HydraServe [7] reduces cold-start latency in public-cloud serving through placement and pipeline parallelism. Snapshot-based warm-up systems [8] pre-materialize process state to bypass portions of startup.

These systems optimize *whether*, *where*, or *when* loading occurs. Tensora asks the complementary question that remains when a load is unavoidable: how should the required reads be submitted? The answer matters in any tiered design because each tier still presents a read plan to userspace.

2.3 Checkpoint formats and loading systems

Checkpoint formats define the request geometry visible to a loader. SafeTensors [9] stores tensor bytes in safe, contiguous files or shards. GGUF targets memory-mapped CPU inference and quantized deployments. ServerlessLLM’s layout [10] distributes tensors across partition files with a central index, enabling tiered or partial placement at the cost of more range-oriented access.

Practical loading infrastructure implements these formats with varying degrees of I/O awareness. PyTorch’s native `torch.load` and `torch.save` provide general tensor serialization but use blocking I/O with no backend selection. Hugging Face’s Accelerate [11] adds model sharding and device placement at the Python level, but the underlying read strategy remains fixed. TensorStore [12] provides a high-performance multi-dimensional array storage library with asynchronous I/O, chunking, and caching, but targets general array access patterns rather than LLM-specific checkpoint loading with format-conditioned backend selection.

Recent systems work attacks loading from other layers. BlitzScale [13] shares already-loaded models across inference instances over high-speed interconnects. PPC/MAIO [14] uses a programmable kernel page cache to adapt prefetching and eviction to LLM I/O patterns. MAIO operates below the system-call interface; Tensora operates above it. The two approaches are complementary: kernel cache policy changes how pages are served, while userspace backend choice changes how reads are issued.

A separate characterization line studies checkpoint/restore I/O patterns on parallel file systems [15], including buffered versus direct paths and liburing. That work focuses on training checkpointing in HPC environments. Tensora focuses on inference-time checkpoint loading in single-node serving and follows the effects through vLLM integration.

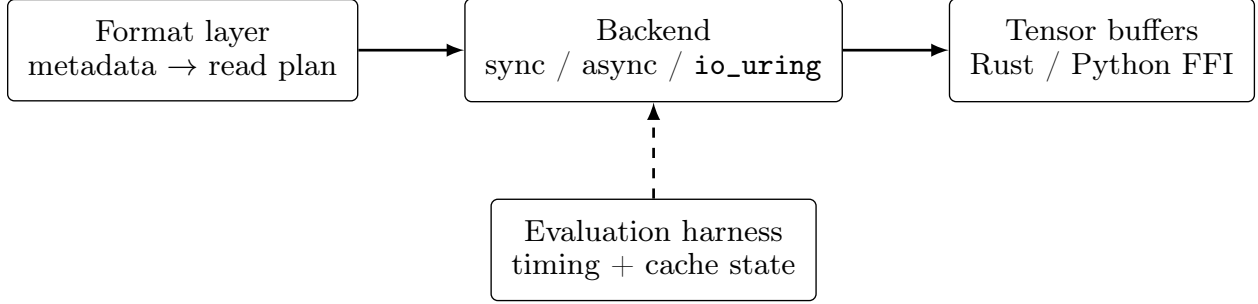


Figure 1: Tensora architecture. The hot path is format parsing → backend execution → tensor buffers. The harness controls timing and cache state outside that path.

2.4 I/O backend selection

The Linux I/O interface landscape includes blocking POSIX calls, async runtimes layered above kernel primitives, POSIX AIO, and `io_uring`. Database research provides the closest methodological warning: Jasny et al. [16] show that `io_uring` is not automatically faster. Its benefit depends on request shape, queue depth, batching, and implementation details. Bulk sequential reads can already amortize syscall overhead; fragmented or high-concurrency reads can benefit from batched submission.

LLM checkpoint loading spans both regimes. SafeTensors resembles contiguous bulk transfer with shard-level fanout. ServerlessLLM-style partitioning resembles grouped range access. Tensora translates the workload-shape lesson to LLM loading and adds three elements missing from prior I/O studies: format-conditioned comparison, cross-layer measurement, and capability-gated fallback when `io_uring` is unavailable in containerized environments.

2.5 Positioning

Tensora is a userspace backend-evaluation and comparison framework for checkpoint loading, not a disaggregated storage system, kernel cache mechanism, or new serving scheduler. MAIO [14] optimizes below the system-call interface; Tensora operates above it. ServerlessLLM [6] optimizes *where* and *when* loading occurs across tiers; Tensora evaluates *how* reads are submitted once loading is unavoidable. The contribution is empirical: backend choice depends jointly on layout, scale, and runtime capability; therefore a loader should expose backend choice matched to workload features rather than assuming a fixed API is optimal.

3 Tensora Design

Tensora is the measurement framework for this paper. Its design goal is to make workload shape, backend choice, and integration overhead explicit enough to measure across I/O backends.

3.1 Design principles

A format parser should produce the same read plan regardless of backend, so timing differences reflect submission mechanism rather than work performed. Backend labels are explicit at the harness level. Measurement (cache drops, timing, capture) stays outside the loader.

3.2 Architecture

Tensora separates format parsing, backend execution, and evaluation (Figure 1). The **format layer** parses checkpoint metadata and produces a request-local read plan: files, byte ranges, tensor names, and expected sizes. The **backend layer** executes reads through an explicit backend (sync, async, or `io_uring`) selected by the caller. The **evaluation layer** contains the Rust profiler and vLLM experiments; Python bindings bridge the integration path.

All backends receive the same read plan from the format layer and execute reads through the selected I/O submission method.

3.3 Common backend surface

All eager backends implement the same conceptual operations:

- `load(path)` for one full file,
- `load_batch(paths)` for multiple whole-file loads,
- `load_range(path, offset, len)` for one byte range, and
- `load_range_batch(requests)` for grouped range reads.

This surface is deliberately small. SafeTensors mostly exercises whole-file and batch loading; Serverless-LLM exercises range batches. Backend implementations may chunk, group, coalesce, or tune queue depth internally, but they must satisfy the same logical tensor plan.

3.4 Backend implementations

The **sync** backend uses blocking POSIX reads with practical host-side parallelism for large files. It is not a strawman serial baseline; its small-model wins are meaningful because it has low orchestration overhead.

The **async** backend is built on Tokio. It groups work per file, batches dynamically, and avoids unnecessary task fanout. It is weak for H100 SafeTensors, where whole-file eager loads do not justify async overhead, but strong for partitioned and network-storage cases.

The **io_uring** backend uses persistent rings, batched submissions, lazy initialization, dynamic ring sizing, and workload-dependent chunk and depth planning. Its value depends on enough queued work; the paper never claims that **io_uring** is intrinsically faster by API name alone.

Table 1: Key design choices and parameters in the evaluated **io_uring** backend.

Aspect	Design
Ring lifetime	Persistent rings with lazy initialization
Submission	Batched SQEs and explicit in-flight management
Completion	Interrupt-driven (default), kernel-poll (SQPOLL with 2s idle timeout, fallback without)
SQ depth	Dynamic per-load: 32 (single file ≤ 512 MB), 64 (≤ 4 GB or small multi-file), 96–128 (large), 256 (very large); clamped to [32, 256]
CQ depth	$2 \times$ SQ depth; clamped to [32, 512]
Flags	SINGLE_ISSUER, COOP_TASKRUN, SUBMIT_ALL (all loads); SQPOLL with fallback to IRQ if unavailable
Chunk sizing	Dynamic: 2MB–512MB per request, governed by alignment, coalescing, and in-flight limits
Fallback	Capability checks exclude io_uring when initialization fails

The mmap-backed path remains useful for lazy selective access, especially through Python handles, but it is not the main eager cold-start mechanism evaluated in the Rust matrices. Eager loading and mmap answer different questions.

3.5 Formats and integrations

Tensora supports two layouts because they expose different regimes. **SafeTensors** presents whole-file or multi-shard contiguous loading. **ServerlessLLM** presents an indexed partition layout with range-oriented reads. Supporting both under one framework is what makes the layout-dependent regime claim measurable.

Python bindings are implemented with PyO3 and expose a small API: sync and async eager load, and lazy `open_*` handles. The async Python path uses a shared Tokio runtime rather than constructing one per call. The vLLM integration uses a custom loader path and follows vLLM’s synchronous loader boundary, so the serving benchmark reflects the interface available to real integrations rather than an invented async serving path. The Python installation is the bridge layer through which vLLM accesses Tensora.

4 Experimental Setup

The evaluation is designed to separate mechanism from deployment impact. A Rust benchmark isolates backend execution. A vLLM benchmark measures the serving-engine boundary; its results include Python integration overhead through Tensor’s Python bindings. The same timing number should not be interpreted the same way across all layers.

4.1 Measurement layers

1. **Rust backend attribution.** Measures checkpoint reading, format handling, and tensor assembly through a selected backend. It excludes Python interpreter costs and serving-engine GPU initialization. This layer answers which submission mechanism is fastest for a fixed logical read plan.
2. **vLLM integration.** Measures loading inside a serving engine, including CUDA context setup, memory pools, runtime configuration, model initialization, and time-to-first-token when requested. This layer answers whether loader choice matters in an end-to-end serving context.

These layers are deliberately not collapsed. A Rust result provides backend evidence; a vLLM result provides deployment evidence. The limitations section states where claims cannot move across layers.

4.2 Research questions

The study answers four questions.

- **RQ1: Backend attribution.** How much does backend choice affect checkpoint loading for real LLM checkpoints?
- **RQ2: Layout dependence.** Does changing checkpoint layout change the backend ranking?
- **RQ3: Serving impact.** Do loader effects survive inside vLLM initialization and TTFT?
- **RQ4: Environment transfer.** Does backend ordering transfer across hardware environments with different storage and kernel configurations?

RQ1 and RQ2 use the Rust matrices, RQ3 uses vLLM, and RQ4 uses cross-environment comparisons.

4.3 Platforms

Measurements were collected on four Linux environments (Table 2). H100 is the primary local-NVMe site. H200 and A100 are network-storage container environments where `io_uring` ring initialization fails with `EPERM`; they test fallback behavior rather than reproducing the exact H100 hardware path. Modal H100 is a gVisor-sandboxed container environment (Section 4.8) used for held-out model validation and full-matrix anchoring; `io_uring` is unavailable (`ENOSYS`) and models are fetched from the HuggingFace CDN on each cold run.

Table 2: Experimental platforms. H100 provides local NVMe and working `io_uring`; H200, A100, and Modal H100 provide restricted environments with blocked `io_uring`.

	H100	H200	A100	Modal H100
GPU	NVIDIA H100	NVIDIA H200	NVIDIA A100 80GB	NVIDIA H100
CPU	x86_64 multi-core	Intel Xeon 8568Y+	AMD EPYC 7543	x86_64 multi-core
RAM	~180 GB	~2 TB	~1 TB	32 GB
Storage	Local NVMe	Network-attached	Network-attached	Ephemeral NVMe
<code>io_uring</code>	Available	Blocked (<code>EPERM</code>)	Blocked (<code>EPERM</code>)	Blocked (<code>ENOSYS</code>)
Kernel	6.x (Ubuntu 24.04)	6.8.0-83 (Ubuntu 22.04)	6.5.0-45 (Ubuntu 22.04)	gVisor

The final software stack used Python 3.12 and Rust 1.94. The optional vLLM dependency is pinned through `bindings/python/uv.lock` (resolved `vllm 0.11.0`). Exports record kernel and package details so that absolute timings can be interpreted with the host context.

4.4 Backends, layouts, and model ladder

The Rust layer evaluates three I/O backends:

- **sync**: blocking POSIX reads with dynamic chunking and host-side parallelism,
- **async**: Tokio-based loading with dynamic batching and per-file grouping,
- **io_uring**: Linux io-uring with persistent rings and workload-driven planning.

The two layouts are **SafeTensors**, a whole-file or multi-shard contiguous layout, and **ServerlessLLM-style partitioning**², an indexed layout accessed through grouped range reads. Memory-mapped APIs are exposed for lazy Python access but are not treated as eager cold-load backends in the Rust matrix.

The model ladder contains six public checkpoints: Qwen/Qwen3-0.6B, HuggingFaceTB/SmolLM3-3B, Qwen/Qwen3-4B, Qwen/Qwen3-8B, Qwen/Qwen3-14B, and Qwen/Qwen3-32B. This ladder spans small, medium, and large checkpoints and includes a non-Qwen model to reduce overfitting to one family. A single model size could justify only a constant backend choice; the ladder is what exposes crossovers.

4.5 Cache state and comparability

Cold-cache behavior is the primary condition for Rust eager-load measurements. A cold-cache run performs:

1. `sync`,
2. `echo 3 > /proc/sys/vm/drop_caches`,
3. benchmark invocation.

Warm-cache conditions skip the cache drop and reuse page-cache state from prior reads. On restricted containers where direct cache dropping is unavailable, the repository-supported page-cache eviction path is recorded as a methodological deviation.

Each backend receives the same logical tensor read plan for a given model and layout. Differences in wall time come from submission strategy, parallelism, chunking, coalescing, and buffer management.

4.6 Baselines and reporting

The Rust matrices compare Tensora backends against one another because the purpose is backend attribution under identical format handling. The vLLM matrix includes a deployment-style baseline: **native** is vLLM’s stock loading path, while loaders prefixed **ts_** use Tensora-backed integrations through the same harness.

Statistical methodology. The H100 Rust evaluation uses a two-tier reporting approach. The main results tables (Tables 5, 6) present representative cold-cache runs for readability. In parallel, 18 of 36 matrix cells (Qwen3-0.6B, Qwen3-8B, and Qwen3-32B across all three backends and both formats) are backed by five-repetition anchor sets with median and range reported in Appendix B.

Where anchor ranges overlap between backends, we avoid forcing a strict ranking and instead state that the difference is not statistically separable at the resolution of five cold-cache repetitions. Separability is assessed by range overlap: if the ranges of two backends overlap, neither can be declared confidently faster. This conservative criterion is appropriate for cold-cache measurements, which have higher variance than warm-cache runs due to cache-state sensitivity. The main claims of the paper rely on large effect sizes and consistent regime patterns (e.g., sync versus io_uring at SafeTensors 8B+, sync’s consistent failure on ServerlessLLM), not on close-call single-cell comparisons.

The Modal full anchor matrix (Section C) provides five-repetition support for all six models across both formats; sync and async ranges overlap at several mid-to-large cells, confirming that close differences in that size range are not statistically separable.

4.7 Falsification criteria

The central claim would be weakened if one fixed explicit backend dominated both layouts and all scales, if sync remained fastest for the largest H100 SafeTensors checkpoints, or if sync became the strongest backend across the ServerlessLLM matrix. It would be refined, not falsified, if another host showed the same regime changes at different model sizes.

²This paper evaluates a partitioned range layout inspired by the ServerlessLLM checkpoint format [10]. The conversion procedure produces index files and partition files matching ServerlessLLM’s structure, but the benchmarks measure Tensora’s own read-planning and backend layers against this layout, not the ServerlessLLM system’s tiered loading pipeline (Section 2).

4.8 Modal H100 environment

A fourth environment uses Modal’s serverless GPU infrastructure for two experiments: held-out model validation (Appendix E) and full-matrix anchoring and close-call analysis (Section C). Each Modal container provisions an H100 GPU with 32 GB RAM and 512 GB ephemeral NVMe. The container runs gVisor (a userspace kernel), which blocks `io_uring` with `ENOSYS`; only sync and async backends are evaluated. Models are downloaded from the HuggingFace CDN on each cold run, and page-cache eviction uses `posix_fadvise(DONTNEED)` rather than `drop_caches`. These methodological differences mean that absolute timings are not directly comparable to the primary H100 results; the Modal experiments test regime transfer and provide statistical support, not identical millisecond reproduction.

5 Results

The evaluation proceeds in two layers: Rust backend attribution (Section 5.2) and vLLM integration (Section 5.5), with Tensora’s Python bindings serving as the bridge between them. Tables present core evidence; figures visualize regime crossovers.

Table 3: Regime map: strongest observed explicit backend by layout, environment, and scale. Color encodes the winning backend. The key visual fact is non-dominance: every backend color appears, and the pattern changes with layout and environment.

Layer / layout	0.6B	4B	8B	32B
H100 SafeTensors	sync	sync	io_uring	io_uring
H100 ServerlessLLM	io_uring	async	io_uring	async
H200 SafeTensors	sync	sync	async	async
H200 ServerlessLLM	async	async	async	async
A100 SafeTensors	sync	async	async	sync
A100 ServerlessLLM	sync	async	async	async
vLLM SafeTensors	sync	sync	io_uring	io_uring

Table 3 is the visual thesis of the paper. SafeTensors begins in the sync regime and shifts with scale. On H100 ServerlessLLM, sync is never the fastest backend. Restricted environments replace `io_uring` with `async`. vLLM preserves the small-versus-large structure. A single fixed backend cannot reproduce this map.

Table 4: Effect-size summary for representative regime claims. Ratios are computed as slower time divided by faster time; larger values indicate stronger evidence for a regime distinction.

Claim	Evidence row	Effect size
Small SafeTensors favors low-overhead sync over <code>io_uring</code>	H100 Qwen3-0.6B	sync is $2.09\times$ faster than <code>io_uring</code>
Large SafeTensors needs deeper submission	H100 Qwen3-32B	<code>io_uring</code> is $1.28\times$ faster than sync
Partitioning removes sync as winner	H100 ServerlessLLM Qwen3-32B	async is $1.36\times$ faster than sync
<code>io_uring</code> availability changes large-scale winner	H200 SafeTensors Qwen3-32B	async is $3.41\times$ faster than sync when <code>io_uring</code> is blocked
Serving impact survives integration	vLLM Qwen3-32B cold load_only	<code>io_uring</code> SafeTensors is 48.1% faster than native

5.1 Headline findings

Four patterns emerge from the Rust and integration matrices. First, SafeTensors exhibits a scale crossover from sync to `io_uring`, while on H100 ServerlessLLM, sync is never the fastest backend. Second, integration overhead reorders but does not erase backend effects: vLLM costs dominate absolute time, yet loader choice still changes cold initialization by tens of seconds at larger scales. Third, ordering regimes transfer better than backend identities: cross-environment validation confirms that small SafeTensors loads favor sync in three of four environments (with async as the gVisor exception on Modal), while the large-scale winner varies

with storage topology and kernel capability. Fourth, no single explicit backend wins across the full parameter space.

5.2 Rust Backend Results

The Rust harness is the authoritative backend-attribution layer because it measures checkpoint loading directly, without Python tensor conversion or serving-engine initialization costs. Unless stated otherwise, each matrix cell is one representative cold-cache run. Half the matrix (18 of 36 cells) is backed by five-repetition anchor sets with median and range (Appendix B). Where anchor ranges overlap between backends, we note that the ranking is not statistically separable rather than forcing a strict order. The main regime claims are supported by non-overlapping ranges and large effect sizes.

5.2.1 SafeTensors Results

Table 5 summarises the explicit backend comparison for SafeTensors.

Table 5: Rust cold-cache SafeTensors results (ms). The crossover from sync-favored small models to io_uring-favored large multi-shard models is the clearest regime signal.

Model	Sync	Async	io_uring
Qwen3-0.6B	555.28	1416.19	1159.24
SmolLM3-3B	1331.79	5802.05	3812.35
Qwen3-4B	1820.85	5039.21	2967.42
Qwen3-8B	3880.40	7074.40	3172.37
Qwen3-14B	7030.25	13489.00	5977.07
Qwen3-32B	15454.45	28763.46	12089.05

Two regimes are visible. The small end of the ladder favors low-overhead sync (555 ms over io_uring’s 1159 ms at Qwen3-0.6B). The ranking flips at Qwen3-8B: io_uring at 3172 ms versus sync at 3880 ms, a $1.22\times$ advantage. At the largest scale (Qwen3-32B), io_uring is $1.28\times$ faster than sync (12089 ms vs 15454 ms). The crossover is not a single threshold; it is a progression visible across the entire ladder.

Anchor data confirms the key claims. At Qwen3-8B, io_uring’s five-repetition range [3160 to 3274 ms] does not overlap sync’s [3648 to 3731 ms], confirming a clean regime separation (non-overlapping five-run ranges). At Qwen3-32B, io_uring [11737 to 14389 ms] is separated from sync [15327 to 16144 ms]. At Qwen3-0.6B the opposite separation is visible: sync [322 to 403 ms] is well below io_uring [1156 to 1264 ms], confirming the small-scale sync regime across all three backends.

Async is consistently the weakest backend for SafeTensors. Whole-file and multi-shard loads benefit from straightforward bulk parallelism rather than scheduler-mediated async orchestration, which adds overhead without corresponding I/O benefits on this workload shape.

Figure 2 visualizes the progression from sync-favored to io_uring-favored loading.

5.2.2 ServerlessLLM-style Partitioned-layout Results

The ServerlessLLM layout produces a different ordering, shown in Table 6.

Table 6: Rust cold-cache ServerlessLLM results (ms). Synchronous loading never wins; the ranking shifts from async at smaller partition scales toward io_uring at larger sizes.

Model	Sync	Async	io_uring
Qwen3-0.6B	1645.06	1471.34	1387.57
SmolLM3-3B	3941.88	2391.24	3091.49
Qwen3-4B	5137.72	3504.25	3727.14
Qwen3-8B	11048.40	9463.37	9043.51
Qwen3-14B	18661.25	16139.08	13257.23
Qwen3-32B	39822.09	29180.89	29779.86

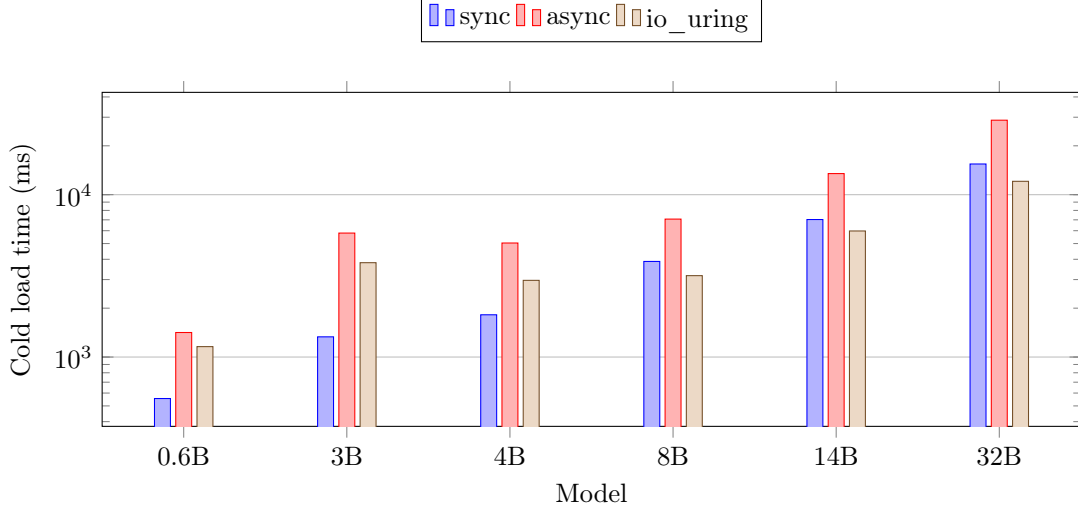


Figure 2: SafeTensors backend crossover (log scale). Sync dominates small models; io_uring dominates large multi-shard models.

Sync is the slowest explicit backend on every ServerlessLLM row, between $1.1\times$ (Qwen3-0.6B) and $1.4\times$ (Qwen3-32B) behind the strongest alternative. The best explicit backend varies with scale: async dominates at smaller partition counts (SmolLM3-3B, Qwen3-4B), while io_uring becomes competitive at larger scales (Qwen3-14B, Qwen3-32B).

Anchor data reveals important nuance. At ServerlessLLM Qwen3-0.6B, async [1331 to 1409 ms] and io_uring [1395 to 1483 ms] have overlapping ranges; the ranking between non-sync backends is not always statistically separable. At Qwen3-8B, io_uring [7119 to 8885 ms] and async [7628 to 9837 ms] overlap; only sync [10175 to 10676 ms] is clearly separated as slowest. At Qwen3-32B, io_uring [26664 to 29152 ms] and async [28639 to 29222 ms] overlap; again, only sync [37785 to 40691 ms] is cleanly separated. The consistent structural fact, that sync loses on every ServerlessLLM row, is robustly confirmed; the ordering among non-sync backends at individual scales is closer and less stable.

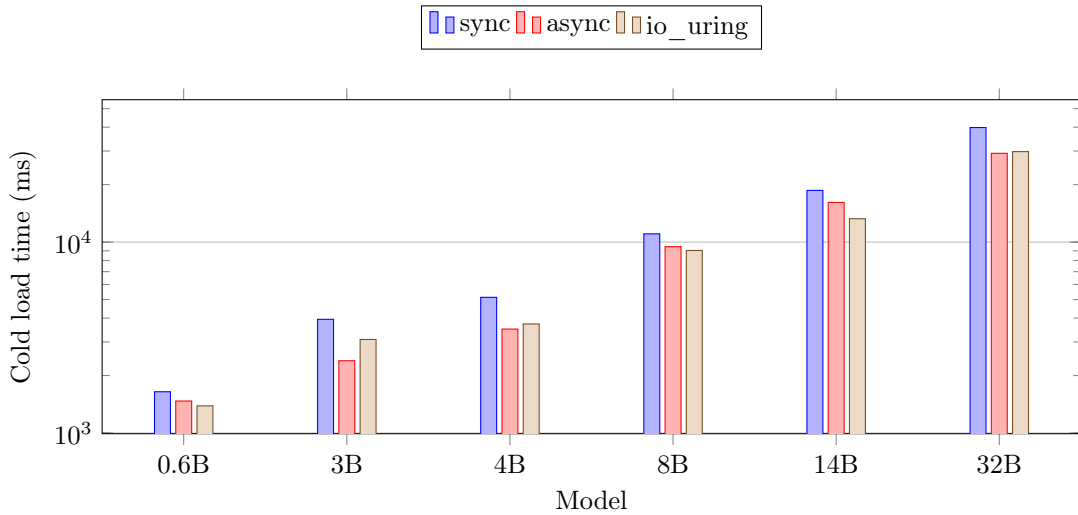


Figure 3: ServerlessLLM backend ordering (log scale). Sync is never the winner; the ranking shifts toward async and io_uring.

Figure 3 shows that the ServerlessLLM workload shape (many small range reads across partition files) rewards different backends than the SafeTensors contiguous regime.

5.3 Backend Non-Dominance Summary

Three findings summarize the backend comparison. First, SafeTensors has a clean crossover: sync dominates at small scale and `io_uring` at large scale on H100. Second, H100 ServerlessLLM removes sync as a winner; async and `io_uring` trade places by partition scale. Third, no single backend wins across both layouts, a direct designer-facing implication.

5.4 Cross-Environment Validation

The H100 results above were collected with all three backends available, including `io_uring`. Two additional environments (an H200 container and an A100 container, both with network-attached storage) block `io_uring` at the kernel level (EPERM). This constraint tests whether ordering regimes transfer across hardware when the strongest H100 backend is unavailable.

The regime structure (backend ordering as a function of format and scale) persists: small SafeTensors loads favor sync on H100, H200, and A100, while on Modal H100 under gVisor async wins at small scale due to gVisor’s per-syscall overhead penalizing synchronous I/O more than batched async submission. The mid-to-large-scale winner changes with storage topology and kernel capability: `io_uring` on local NVMe (H100), async on network storage (H200), and sync recovering at the largest A100 scale. Complete cross-environment matrices appear in Appendix D.

Held-out validation on Mistral-7B-Instruct (Appendix E), a different architecture family with different shard geometry, confirms that the structural regime claims generalize beyond the primary model ladder: sync is never fastest for ServerlessLLM on either model. The SafeTensors pattern is geometry-dependent: async dominates the 5-shard Qwen3-8B by $2.92\times$, while sync remains fastest on Mistral-7B’s 3-shard layout, consistent with the finding that low-shard-count SafeTensors loads favor low-overhead backends.

5.5 vLLM Results

The final vLLM matrix completed successfully across all six models, all benchmark loaders, warm and cold cache conditions, and all three benchmark kinds.

Stock vLLM versus Tensora. The `native` loader follows vLLM’s default weight-loading path; `ts_*` loaders use Tensora-backed integrations. Table 7 shows the complete cold `load_only` matrix across all six models. On cold `load_only`, Tensora’s SafeTensors loader family reduces initialization time relative to `native` across all scales.

Table 7: Cold vLLM `load_only` across all models and loader families (ms). Tensora’s SafeTensors explicit backends consistently outperform native. ServerlessLLM paths improve over native on large models.

Model	<code>native</code>	<code>ts_st_sync</code>	<code>ts_st_iour</code>	<code>ts_sllm_sync</code>	<code>ts_sllm_iour</code>
Qwen3-0.6B	23659	20417	20867	21191	21262
SmolLM3-3B	36874	27767	29992	30870	30287
Qwen3-4B	40486	28478	29727	32327	31792
Qwen3-8B	56639	36524	35889	44371	42343
Qwen3-14B	89220	49435	47751	63150	61848
Qwen3-32B	169971	92421	88164	124049	111717

The improvement grows with model size: at Qwen3-32B, Tensora’s SafeTensors `io_uring` reduces initialization by 48.1% relative to native (88164 ms vs 169971 ms). This is consistent with the Rust SafeTensors finding that `io_uring`’s advantage grows with scale. The ServerlessLLM paths also improve over native at large scales (34% reduction at 32B) but trail SafeTensors paths by 21 to 34%, consistent with the Rust finding that partitioned range layouts have higher overhead.

For cold `load_only` across all models, the best loader varies with model size. Table 8 shows the strongest loader in the final cold-load-only configuration.

The vLLM best-loader pattern mirrors the Rust SafeTensors crossover: smaller models favor sync, larger models favor `io_uring`. The match is not exact (vLLM initialization includes fixed costs that dilute the pure-I/O ranking), but the regime structure survives.

Table 8: Best cold load_only vLLM loader by model. The pattern follows the Rust SafeTensors regime: sync at small scales, io_uring at large scales.

Model	Best init time (ms)	Best loader
Qwen3-0.6B	20417	ts_safetensors_sync
SmolLM3-3B	27767	ts_safetensors_sync
Qwen3-4B	28478	ts_safetensors_sync
Qwen3-8B	35889	ts_safetensors_io_uring
Qwen3-14B	47751	ts_safetensors_io_uring
Qwen3-32B	88164	ts_safetensors_io_uring

For TTFT, the ranking follows load_only but the proportional advantage shrinks. At Qwen3-32B, ts_safetensors_io_uring TTFT (89259 ms median) is 41.3% lower than native (152184 ms median); the load_only reduction at the same model was 48.1%. The smaller TTFT improvement reflects the first-generation compute step diluting the I/O savings. The SafeTensors loader family consistently outperforms ServerlessLLM for both metrics. Load_only and TTFT show backend effects mainly in initialization; decode is comparatively insensitive once weights are resident.

5.6 Layer summary

Four principles emerge. (1) Backend ranking depends on workload shape: SafeTensors favors sync at small scale and io_uring at large H100 scale, while on H100 ServerlessLLM, sync is never the fastest backend. (2) Integration layers reorder but do not eliminate backend differences: vLLM adds engine initialization cost that compresses proportional spreads, yet backend choice still changes user-visible startup. (3) Cross-environment validation confirms that small SafeTensors loads favor sync on H100, H200, and A100 (with Modal H100 as the gVisor exception where async wins at small scale); the mid-to-large-scale winner varies with kernel interface availability and storage topology. (4) No single explicit backend dominates the full parameter space; loader design should therefore expose backend choice and regime awareness.

6 Discussion

The main result is not that one backend wins. The main result is that backend choice depends jointly on format, scale, environment, and observation layer.

6.1 Mechanism: why layout changes the answer

SafeTensors and ServerlessLLM-style partitioning create different I/O problems. SafeTensors exposes large contiguous reads after metadata parsing. For small checkpoints, a practical sync backend has low overhead and enough parallelism; additional scheduling machinery does not amortize. As size and shard count grow, the H100 workload has enough in-flight work for io_uring to pay off.

ServerlessLLM-style partitioning changes the request geometry. The loader reconstructs tensors from indexed partition files, so grouping, coalescing, and range batching dominate. The SafeTensors rule “sync can be good for small loads” does not transfer: sync never wins in the H100 ServerlessLLM matrix. The format, not the API name, explains the reversal.

6.2 Integration: why rankings move across layers

Rust and vLLM answer different questions. Rust isolates backend mechanics. vLLM measures serving startup with CUDA context setup, memory pools, runtime configuration, and generation setup; its path includes Python integration overhead through Tensor’s Python bindings. These costs can dilute a pure I/O advantage or reorder two close backends.

The vLLM results do not reproduce Rust rankings row-for-row because vLLM adds substantial initialization overhead that compresses proportional backend spreads. For SafeTensors Qwen3-8B, the Rust times span 3.2 to 7.1 s (a $2.2\times$ spread), while the vLLM cold times span 35.9 to 36.5 s (a $1.02\times$ spread). The I/O component becomes a small fraction of total startup. Yet the regime structure survives: sync-type loaders win at small

scale and `io_uring`-type loaders win at large scale in both Rust and vLLM. The proportional advantage is smaller at the vLLM level, but the absolute savings (21 s at 8B, 82 s at 32B) are deployment-relevant.

This is not a flaw in the methodology; it is the deployment reality. A systems paper about loading should show both mechanism and propagation. Backend effects matter because they remain visible in vLLM cold initialization even after fixed engine costs are included.

Warm-cache behavior adds a second dimension. vLLM’s native initialization is slower on warm cache than cold (0.6B: 23659 ms cold vs 56874 ms warm), indicating its bottleneck is CPU processing, not I/O. Tensor’s I/O-gated loaders are faster on warm cache (`ts_safetensors_sync` at 0.6B: 20417 ms cold vs 18223 ms warm). This asymmetry means backend-aware loading provides more predictable restart times regardless of cache state.

6.3 What transfers

Absolute milliseconds are machine-specific. Ordering regimes transfer more plausibly. In this study, small SafeTensors loads favor sync on H100, H200, and A100 (with Modal H100 as the gVisor exception where async leads at small scale). The large-scale winner does not transfer by name: H100 favors `io_uring`, H200 favors async when `io_uring` is blocked, and A100 shows a non-monotonic recovery at the largest SafeTensors row. A replication should therefore ask whether regime structure (backend ordering as a function of format and scale) persists, not whether every number matches.

6.4 A100 SafeTensors at 32B: why sync recovers

The A100 SafeTensors result at 32B presents a distinctive reversal: sync (46,923 ms) outperforms async (60,807 ms) by 1.30 $\times$. This contrasts with H200 at the same scale, where async dominates sync by 3.41 $\times$. One plausible explanation is storage-link saturation: at 65 GB, the checkpoint transfer may saturate the A100’s network-attached storage bandwidth, causing async’s Tokio scheduling overhead to exceed its concurrency benefit. The recovery does not appear on H200, where the storage path sustains higher effective bandwidth. This finding illustrates why storage topology should inform backend choice.

6.5 Implications

For practitioners, evaluate backend choice relative to format and scale rather than adopting a fixed submission policy. Expose explicit backend labels for known regime boundaries.

For researchers, report loading results as layered constructs. A Rust microbenchmark and a serving-engine startup benchmark are both valuable; neither subsumes the other. A paper that reports only one layer risks explaining the wrong phenomenon.

7 Limitations

The paper makes a bounded systems claim: backend choice should depend on workload shape. It does not prove a universal selector, universal thresholds, or universal superiority of any I/O API.

The primary matrices come from one H100 host with local NVMe and working `io_uring`. H200 and A100 provide useful but different validation: network-storage containers where `io_uring` is blocked. They test regime transfer, but they are not three identical replicas of the same hardware/storage stack. The model ladder covers six public checkpoints from 0.6B to 32B parameters and two layouts. It does not cover GGUF, quantized-only deployments, object stores, distributed filesystems, RAID variants, direct-I/O configurations, Windows/macOS backends, or multi-node distributed loading. “Load time” also means different things at different layers: Rust timings approximate backend-attributed disk-to-tensor work; vLLM timings add Python integration overhead, engine initialization, and CUDA setup. A Rust result should not be quoted as a serving result.

The results compare three specific implementations (sync POSIX, Tokio async, and `io_uring`), not the best possible implementation of each API. A different sync backend with adaptive parallelism or a differently tuned `io_uring` setup might shift the crossover points. This does not invalidate the central claim, that checkpoint loading has identifiable regimes that shift with layout and scale, because that claim depends on the workload-shape explanation (contiguous vs. partitioned reads), not on any single backend being universally optimal.

Half of the H100 Rust matrix (18 of 36 cells) is backed by five-repetition anchor sets; the remaining cells are representative cold-cache runs. Repeating the full H100 matrix for complete variance estimation would strengthen the statistical base; the Modal repeated anchor matrix (five repetitions across all six models and both formats) provides the closest available proxy. The paper’s main claims rest on separable regime distinctions (SafeTensors sync-to-`io_uring` crossover, ServerlessLLM sync-never-wins), confirmed by non-overlapping anchor ranges. Close-call rankings are explicitly flagged as not statistically separable. No explicit backend dominates across layout, scale, and environment. Held-out validation on Mistral-7B-Instruct confirms regime transfer but also shows that effect sizes vary with shard geometry; broader held-out coverage would further test generalization.

The Python integration layer is not independently benchmarked in this study. The Rust-to-vLLM stack provides end-to-end evidence that backend effects propagate through the serving stack, but the Python-level contribution is not separately quantified.

8 Conclusion

Checkpoint loading for LLM serving exhibits identifiable performance regimes: the optimal I/O backend changes systematically with checkpoint format, model scale, and deployment environment. This paper presents a controlled evaluation of three I/O backends: sync, async, and `io_uring`, across two checkpoint layouts, two measured layers, six model scales, and four environments. H100 SafeTensors shifts from sync-favored small loads to `io_uring`-favored large loads, with `io_uring` providing a $1.28\times$ advantage at 32B. On H100 ServerlessLLM, sync is never the fastest backend. vLLM integration reorders the absolute rankings relative to Rust but preserves the practical consequence: properly selected backends yield measurable improvements, with `io_uring` SafeTensors reducing vLLM cold initialization by 37 to 48% at 8B and 32B. Cross-environment measurements show that regime structure (small-scale sync dominance, large-scale variation) transfers better than backend identity. These findings suggest that checkpoint loading should be treated as a workload-dependent I/O evaluation problem rather than a fixed initialization cost.

A Reproducibility

This appendix describes how to rebuild the paper and reproduce the ordering evidence. The goal is traceability and comparable regimes, not identical milliseconds on different hardware.

A.1 Paper build

Build from `paper/`:

```
pdflatex -interaction=nonstopmode main.tex
bibtex main
pdflatex -interaction=nonstopmode main.tex
pdflatex -interaction=nonstopmode main.tex
```

The public source bundle should include `main.tex`, `template.tex`, `PRIMEarxiv.sty`, `sections/`, `figures/`, `library.bib`, and `OOREADME.json`. If a venue template replaces the arXiv style, rebuild with that venue’s class and rules.

A.2 Repository and revision

The implementation is open source at <https://github.com/botirk38/tensora>. Record the exact revision next to exported results:

```
git rev-parse HEAD
git status -sb
```

Also record `date -u`, `uname -a`, `rustc -version`, `python3 -version`, GPU and driver information from `nvidia-smi`, and the active vLLM package version when vLLM is used.

A.3 Rust profiling

Build the profiler in release mode:

```
cargo build --release --bin profile
```

Run one cell with:

```
cargo run --release --bin profile -- <format> <backend> \
  --model-id <MODEL_ID> --iterations 1
```

where `<format>` is `safetensors` or `serverlessllm`, and `<backend>` is `sync`, `async`, or `io-uring`. SafeTensors uses the Hugging Face Hub cache. ServerlessLLM-style conversions are stored under the Tensora cache directory, for example `$XDG_CACHE_HOME/tensora/<slug>/serverlessllm`.

For cold-cache runs on hosts with sufficient privileges, run `sync` followed by `echo 3 > /proc/sys/vm/drop_caches` before each invocation. On restricted containers, use the repository-supported eviction path and report that deviation. The full H100 matrix in `results/h100/rust.tsv` was generated by sweeping model ids, formats, and backends with that ritual.

A.4 Python and vLLM harnesses

Python benchmarks live under `bindings/python/benchmarks/`. The convenience script `scripts/run\_benchmarks.sh` runs pytest-benchmark suites and writes JSON outputs. Set `TENSORA_BENCH_MODELS` to one or more Hugging Face model ids, separated by spaces or commas. Set `TENSORA_BENCH_NO_VLLM=1` when a GPU-backed vLLM environment is unavailable.

When vLLM is enabled, runs are serialized by model to avoid GPU memory interference. The paper’s archived exports live under `results/h100/`, `results/h200/`, and `results/a100/`. The H100 vLLM matrix in `results/h100/vllm.tsv` sweeps loaders, cache modes, and benchmark kinds: `load_only`, `TTFT`, and `steady_state_decode`.

A.5 What a replication should check

A strong replication should check ordering regimes before exact milliseconds:

- SafeTensors changes backend preference with scale.
- ServerlessLLM-style partitioning does not follow the SafeTensors sync-first rule.
- vLLM initialization is sensitive to loader choice and includes Python integration overhead from the binding layer, while steady-state decode is much less sensitive.
- Blocking `io_uring` changes large-scale winners.

B Cold-cache anchor statistics

Tables 9 and 10 summarize five cold-cache repetitions for all 18 anchored H100 cells from `results/h100/anchor.tsv`. Anchors cover Qwen3-0.6B, Qwen3-8B, and Qwen3-32B across all three backends and both formats (18 of 36 total matrix cells). The main result tables use one representative cold run per cell for readability; these anchors provide statistical support.

The *Sep.* column indicates whether the backend’s range is cleanly separated from the next-closest backend at the same model/format (✓ = non-overlapping, ○ = overlapping with at least one comparator). Where ranges overlap, the ranking should not be treated as strict.

Key observations from anchor analysis. The anchors confirm the paper’s main structural claims with non-overlapping ranges: (1) SafeTensors `io_uring` dominates `sync` at 8B and 32B, (2) `sync` dominates `io_uring` at 0.6B, and (3) `sync` is the slowest ServerlessLLM backend at all anchored scales. Within ServerlessLLM, the ordering among `async` and `io_uring` is not robustly separable at any individual scale; the consistent finding is that both outperform `sync`, not that either one dominates.

C Modal anchor statistics

A five-repetition matrix was collected on Modal H100 containers (gVisor, `io_uring` blocked). This matrix covers all six models across `sync` and `async` backends, with close-call analysis derived from this matrix. Table 11 and Table 12 summarize the SafeTensors and ServerlessLLM results.

Table 9: SafeTensors anchor repetitions: median (ms) with range across five cold-cache runs.

Model	Backend	Median	Range	CV%	Sep.
Qwen3-0.6B	sync	323	322 to 403	9.1	✓
Qwen3-0.6B	io_uring	1 159	1 156 to 1 264	3.6	✓
Qwen3-0.6B	async	1 735	1 572 to 1 792	4.6	✓
Qwen3-8B	io_uring	3 250	3 160 to 3 274	1.3	✓
Qwen3-8B	sync	3 701	3 648 to 3 731	0.8	○
Qwen3-8B	async	6 805	6 510 to 7 161	3.3	✓
Qwen3-32B	io_uring	12 204	11 737 to 14 389	7.6	✓
Qwen3-32B	sync	15 462	15 327 to 16 144	2.0	○
Qwen3-32B	async	26 685	25 868 to 27 416	1.9	✓

Table 10: ServerlessLLM anchor repetitions: median (ms) with range across five cold-cache runs.

Model	Backend	Median	Range	CV%	Sep.
Qwen3-0.6B	async	1 353	1 331 to 1 409	2.1	○
Qwen3-0.6B	io_uring	1 417	1 395 to 1 483	2.2	○
Qwen3-0.6B	sync	1 456	1 409 to 1 524	3.0	○
Qwen3-8B	io_uring	7 267	7 119 to 8 885	8.8	○
Qwen3-8B	async	9 172	7 628 to 9 837	10.0	○
Qwen3-8B	sync	10 525	10 175 to 10 676	1.9	✓
Qwen3-32B	io_uring	27 910	26 664 to 29 152	2.9	○
Qwen3-32B	async	29 011	28 639 to 29 222	0.8	○
Qwen3-32B	sync	39 241	37 785 to 40 691	2.4	✓

Table 11: Modal H100 SafeTensors anchor repetitions: median (ms) with range across five cold-cache runs.

Model	Backend	Median	Range	CV%	N
Qwen3-0.6B	async	558	518–635	8.2	5
Qwen3-0.6B	sync	713	675–724	3.0	5
SmolLM3-3B	async	2 339	2 120–2 547	6.6	5
SmolLM3-3B	sync	2 602	1 647–2 832	19.5	5
Qwen3-4B	async	2 908	2 764–3 131	5.4	5
Qwen3-4B	sync	3 182	1 813–3 232	21.2	5
Qwen3-8B	async	5 378	3 576–6 267	22.2	5
Qwen3-8B	sync	6 496	3 781–6 502	20.2	5
Qwen3-14B	async	9 910	6 681–10 748	21.7	5
Qwen3-14B	sync	10 913	6 647–11 189	19.2	5
Qwen3-32B	async	20 871	14 418–23 550	21.7	5
Qwen3-32B	sync	23 314	15 191–24 039	17.1	5

Table 12: Modal H100 ServerlessLLM anchor repetitions: median (ms) with range across five cold-cache runs.

Model	Backend	Median	Range	CV%	<i>N</i>
Qwen3-0.6B	async	1 171	1 118–1 206	3.3	5
Qwen3-0.6B	sync	1 303	1 068–1 402	9.9	5
SmolLM3-3B	async	4 569	4 263–4 684	3.5	5
SmolLM3-3B	sync	4 741	4 272–5 221	7.1	5
Qwen3-4B	async	6 233	5 494–7 058	9.4	5
Qwen3-4B	sync	6 563	5 138–7 079	11.9	5
Qwen3-8B	async	12 808	10 835–13 579	8.4	5
Qwen3-8B	sync	13 382	11 369–13 872	8.4	5
Qwen3-14B	sync	22 816	21 697–24 871	5.4	5
Qwen3-14B	async	23 069	22 043–23 327	2.4	5
Qwen3-32B	async	51 720	47 340–56 205	6.1	5
Qwen3-32B	sync	53 971	28 707–71 612	38.4	5

Key observations. (1) Sync is the slowest or near-slowest ServerlessLLM backend at all Modal scales, replicating the primary H100 finding. (2) SafeTensors at 0.6B shows a clean separation: async (558 ms) < sync (713 ms); without `io_uring`, async becomes the small-scale winner rather than sync. (3) Most mid-to-large SafeTensors and ServerlessLLM cells show overlapping ranges among non-sync backends, consistent with the H100 observation that close-call rankings are noise-dominated. (4) CDN-induced variance (Qwen3-32B sync CV=38%) inflates ranges on Modal relative to local-NVMe H100; regime structure is preserved, but individual-cell precision is lower.

Close-call analysis. The full anchor matrix confirms range overlap at the mid-to-large cells where backend differences narrow. SafeTensors Qwen3-14B: sync [6 647 to 11 189 ms] vs. async [6 681 to 10 748 ms] (**OVERLAP**). ServerlessLLM Qwen3-14B: sync [21 697 to 24 871 ms] vs. async [22 043 to 23 327 ms] (**OVERLAP**). ServerlessLLM Qwen3-4B: sync [5 137 to 7 079 ms] vs. async [5 494 to 7 058 ms] (**OVERLAP**). These confirm that the differences at those cells are not statistically separable, consistent with the paper’s conservative separation criterion.

D Cross-environment validation

This section presents the full cross-environment matrices referenced in Section 5.4. Results are collected on H200 and A100 container environments where `io_uring` is blocked at the kernel level (EPERM).

D.1 H200 Results (`io_uring` blocked)

Table 13 shows the H200 SafeTensors matrix. With `io_uring` blocked, async replaces it as the fastest backend at large scale, while sync remains dominant for small models.

Table 13: H200 cold-cache SafeTensors results (ms). `io_uring` is blocked (EPERM). Sync dominates at small scale; async takes over at large scale.

Model	Sync	Async
Qwen3-0.6B	604.34	852.17
SmolLM3-3B	2013.66	2938.30
Qwen3-4B	909.83	2478.10
Qwen3-8B	3924.69	2977.59
Qwen3-14B	25872.03	4767.67
Qwen3-32B	36205.91	10623.83

The H200 SafeTensors crossover preserves the structural pattern from H100: sync wins at small scale (0.6B to 4B), while a different backend dominates at large scale (8B+). On H100 that backend is `io_uring`; on H200 it is async.

Table 14 shows the H200 ServerlessLLM matrix.

Table 14: H200 cold-cache ServerlessLLM results (ms). Async dominates all rows.

Model	Sync	Async
Qwen3-0.6B	1565.27	1491.44
SmolLM3-3B	3945.68	1947.39
Qwen3-4B	5100.25	2489.27
Qwen3-8B	10788.88	6045.47
Qwen3-14B	17765.28	14450.40
Qwen3-32B	39061.57	27367.74

The ServerlessLLM ordering on H200 matches H100 structurally: sync is never the fastest, and async dominates all rows.

D.2 A100 Results (`io_uring` blocked, reduced ladder)

The A100 environment runs a four-model reduced ladder (0.6B, 4B, 8B, 32B) with the same EPERM constraint. Table 15 and Table 16 summarise the results.

Table 15: A100 cold-cache SafeTensors results (ms). Network-attached storage with AMD EPYC CPU. Sync wins at 0.6B; async dominates at 4B to 8B; sync recovers at 32B.

Model	Sync	Async
Qwen3-0.6B	724.26	1094.43
Qwen3-4B	7598.41	3532.11
Qwen3-8B	8412.36	4235.21
Qwen3-32B	46923.42	60807.34

Table 16: A100 cold-cache ServerlessLLM results (ms). Sync and async are essentially tied at 0.6B; async dominates 4B and above.

Model	Sync	Async
Qwen3-0.6B	2183.33	2250.91
Qwen3-4B	7551.79	4486.75
Qwen3-8B	14707.01	11528.26
Qwen3-32B	56025.98	40472.54

The A100 SafeTensors results show a distinctive pattern: sync wins at 0.6B, async dominates the mid-range (4B to 8B), but sync recovers at 32B. This contrasts with both H100 (where `io_uring` dominates large scale) and H200 (where async dominates large scale). For ServerlessLLM, sync and async are essentially tied at 0.6B (2183 ms vs 2251 ms); async dominates from 4B upward.

D.3 Cross-Environment Summary

Table 17 summarises the best explicit backend per row across all four environments.

Four findings from the cross-environment comparison: (1) Sync dominates small SafeTensors models in three of four environments; on Modal (gVisor), async wins even at 0.6B because `io_uring` is blocked and the gVisor syscall path penalizes synchronous reads. (2) At large scale, the strongest backend depends on available kernel interfaces and storage topology: `io_uring` on local NVMe (H100), async on network storage (H200 and Modal), and sync on a different network-storage configuration (A100 at 32B). (3) Small-scale ordering transfers better than large-scale ordering: the small-scale winner is consistent across H100, H200, and A100, while the large-scale winner varies. (4) Modal H100 adds a fourth datapoint: the regime structure (non-sync backends dominate at mid-to-large scale) persists under gVisor sandboxing with ephemeral storage and CDN-based model fetching.

Table 17: Best explicit backend by environment for SafeTensors. The crossover from sync to another backend persists across all four environments. Which alternative wins at mid-to-large scale depends on hardware, storage topology, and kernel interface availability.

Model	H100	H200	A100	Modal H100
Qwen3-0.6B	sync	sync	sync	async
Qwen3-4B	sync	sync	async	async
Qwen3-8B	io_uring	async	async	async
Qwen3-32B	io_uring	async	sync	async

E Held-out model validation

The primary model ladder is drawn from a single checkpoint family (Qwen3, supplemented by SmolLM3-3B). To test whether the regime structure generalises beyond this geometry, two held-out models not used in threshold derivation were evaluated on Modal H100 containers (`io_uring` unavailable).

Table 18: Held-out model validation on Modal H100 (ms, `io_uring` blocked). Mistral-7B is a different architecture family with 3 shards and 582 tensors.

Model	Format	Sync	Async
Qwen3-8B	SafeTensors	11 330	3 884
Qwen3-8B	ServerlessLLM	10 662	5 184
Mistral-7B	SafeTensors	6 889	8 596
Mistral-7B	ServerlessLLM	24 195	21 888

Two regime findings transfer. First, **sync is never the fastest ServerlessLLM backend** for either held-out model: async beats sync by $2.06\times$ on Qwen3-8B and by $1.11\times$ on Mistral-7B. The structural claim that partitioned layouts disfavor synchronous submission holds across both checkpoint families. Second, **SafeTensors at 8B scale favours non-sync backends** when `io_uring` is unavailable: async is $2.92\times$ faster than sync for Qwen3-8B SafeTensors, consistent with the H200 finding that async replaces `io_uring` as the large-scale winner in restricted environments. Mistral-7B SafeTensors shows a different pattern: sync is fastest (6 889 ms vs. async 8 596 ms), consistent with its 3-shard geometry reducing the concurrency benefit that async exploits on the 5-shard Qwen3-8B.

Mistral-7B’s ServerlessLLM spread is narrower ($1.11\times$) than Qwen3-8B’s ($2.06\times$). This is consistent with Mistral’s different shard geometry (3 shards of ~ 9.7 GB each vs. Qwen3-8B’s 5 shards of ~ 3.3 GB), which reduces the concurrency advantage that async exploits. The regime structure transfers; the effect size is geometry-dependent.

F Artifact layout

Each results directory contains tab-separated summaries and raw exports. `results/<env>/rust.tsv` contains Rust cold-cache profiles. `results/<env>/vllm.tsv` contains vLLM integration benchmarks where available. `results/<env>/anchor.tsv` contains repeated anchors where collected. `results/<env>/raw/` contains pytest-benchmark JSON exports.

Archived exports may include exploratory or fallback rows (e.g., rows with `default` backend label from earlier versions of the study). These are retained for provenance but are not referenced in the paper’s main comparison. The paper reports only the three explicit backends (sync, async, `io_uring`) as described in Section 4.

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023)*. ACM, 2023.

- [2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.
- [4] Reza Yazdani Aminabadi et al. DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2022)*. IEEE/ACM, 2022.
- [5] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs, 2024.
- [6] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.
- [7] Anonymous Authors. HydraServe: Minimizing cold start latency for serverless LLM serving in public clouds. arXiv preprint arXiv:2502.15524, 2025.
- [8] Paulo Silva and Thiago Emmanuel Pereira. Performance evaluation of snapshot methods to warm the serverless cold start, 2021.
- [9] Hugging Face. Safetensors documentation. <https://huggingface.co/docs/safetensors>, 2024. Accessed: April 2026.
- [10] ServerlessLLM Contributors. Serverlessllm. <https://github.com/ServerlessLLM/ServerlessLLM>, 2024. GitHub repository.
- [11] Hugging Face. Accelerate: A library for easy mixed precision and distributed training. <https://huggingface.co/docs/accelerate>, 2024. Checkpoint loading and model sharding utilities.
- [12] Google. Tensorstore: High-performance multi-dimensional array storage. <https://google.github.io/tensorstore/>, 2024. Accessed: April 2026.
- [13] Zizhao Mo et al. Blitzscale: Efficient large model serving with elastic GPU resource sharing. In *Proceedings of the 2024 USENIX Annual Technical Conference (ATC 2024)*. USENIX Association, 2024.
- [14] Yubo Liu, Hongbo Li, Xiaojia Huang, Yongfeng Wang, Hanjun Guo, Hui Chen, Yuxin Ren, and Ning Jia. Accelerating model loading in LLM inference by programmable page cache. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST 2026)*. USENIX Association, 2026.
- [15] Nicolas Pretet et al. Understanding LLM checkpoint/restore I/O strategies and patterns, 2025.
- [16] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-performance DBMSs with io_uring: When and how to use it, 2025.